WIOT | Power Modes Reference

Version 1.0 | 04 October 2026 | Arduino STM32F401 node and LoRa gateway

01 | What is available now?

This reference describes the implemented firmware, not a proposed design. It is based on firmware revision `48e5e4b`. The STM32CubeIDE and ESP32 ports are outside its scope. No runtime code, private configuration or deployed setting is modified by this documentation update.

Yes: the controller can sleep while the selected modem stays NORMAL. Choose REMOTE_WAKE. This is light CPU Sleep, not deep Standby. SIM800, ESP8266 and LoRa each support this profile.

Firmware mode	Controller between work cycles	Selected radio	Server command while idle	RAM on wake
NORMAL	Runs the normal application loop	NORMAL	Handled while connected	Retained
RTC_PERIODIC	STM32 Standby	ESP deep sleep or LoRa Mode 3	No reception in Standby; use awake window	Lost; setup runs again
REMOTE_WAKE	CPU Sleep / WFI between service passes	NORMAL and serviced	Yes, while the link is available	Retained
RTC_QUEUED	STM32 Standby	ESP deep sleep or LoRa Mode 3	Queue on server; deliver after next Status	Lost; setup runs again
LORA_WOR_STOP	STM32 Stop, with RTC heartbeat	LoRa Mode 2, wake-on-radio	Yes, via an awake WOR-enabled gateway	Retained

Three different meanings of sleep

- **CPU Sleep:** CPU waits for an interrupt; clocks, UART and the 1-ms tick remain active. REMOTE_WAKE pauses scheduled sensor work, but keeps communication and the watchdog serviced. It is not continuous low-current Stop.
- **Stop:** LORA_WOR_STOP suspends the tick and enters low-power Stop. RTC or LoRa AUX wakes the CPU; board clocks and UART are restored before normal work resumes.
- **Standby:** RTC_PERIODIC and RTC_QUEUED enter deep controller sleep. Wake is a reset, not a return to the previous function. A controlled software-reset handoff enters Standby before the software-started watchdog is armed.

Practical limits

The supported combinations are 11 sensor-node profiles and 2 gateway hardware combinations. Each gateway combination can enable or disable WOR transmission: **15 practical profiles in total**. Unselected modules are not extra active transports and are not automatically powered off. The gateway must remain NORMAL.

There is no implemented SIM800 modem-sleep profile, no controller-Standby-plus-normal-modem profile, and no generic server wake from full Standby. These are firmware/PCB restrictions, not statements that such designs are physically impossible.

02 | Complete compatibility matrices

YES means accepted by the current configuration guards for the active transport. NO means rejected. Keep unused component policies NORMAL; changing an unused policy is not an additional useful power-saving combination.

Sensor node with direct SIM800 MQTT

Controller profile	SIM NORMAL	SIM POWER_DOWN
NORMAL	YES	NO
RTC_PERIODIC	NO	NO
REMOTE_WAKE	YES	NO
RTC_QUEUED	NO	NO
LORA_WOR_STOP	NO	NO

Sensor node with direct ESP8266 MQTT

Controller profile	ESP NORMAL	ESP POWER_DOWN
NORMAL	YES	NO
RTC_PERIODIC	NO	YES
REMOTE_WAKE	YES	NO
RTC_QUEUED	NO	YES
LORA_WOR_STOP	NO	NO

Sensor node with LoRa-to-gateway uplink

Controller profile	LoRa NORMAL	LoRa POWER_DOWN	LoRa WAKE_ON_RADIO
NORMAL	YES	NO	NO
RTC_PERIODIC	NO	YES	NO
REMOTE_WAKE	YES	NO	NO
RTC_QUEUED	NO	YES	NO
LORA_WOR_STOP	NO	NO	YES

Gateway: all combinations

Only controller NORMAL + LoRa NORMAL + selected IP modem NORMAL is accepted, for either SIM800 or ESP8266. Every other controller/radio combination is rejected. For either accepted combination, `APP_LORA_GATEWAY_WOR_TX` may be 0 or 1. The value 1 adds wake-probe transmission; it does not put the gateway to sleep.

Independent macros do not mean unrestricted combinations. A sleeping radio cannot receive a server command, and this PCB does not provide an implemented safe SIM shutdown/restart and remote interrupt-wake path. The guards deliberately reject those combinations.

03 | All supported configuration recipes

Set the existing macros in `Node/IoT_Node_Encrypted/src/config/AppConfig.h`, then compile and flash. Do not add a second conflicting definition. For IP builds choose exactly one of `USE_SIM800` or `USE_ESP8266`. If neither is defined, this header currently defaults to SIM800.

Numeric key for the recipe table

`ROLE`: 1 = sensor node, 2 = gateway. `UP`: 1 = MQTT direct, 2 = LoRa to gateway. `CTRL`: 0 = NORMAL, 1 = RTC_PERIODIC (POWER_DOWN), 2 = REMOTE_WAKE, 3 = LORA_WOR_STOP (LORA_WOR), 4 = RTC_QUEUED (QUEUED). `L/W/S`: LoRa / WiFi / SIM policy; 0 = NORMAL, 1 = POWER_DOWN; LoRa alone also accepts 2 = WAKE_ON_RADIO. `WTX` is the gateway WOR switch.

ID	ROLE	UP	IP selection	CTRL	L	W	S	WTX
N-SIM	1	1	SIM800	0	0	0	0	0
R-SIM	1	1	SIM800	2	0	0	0	0
N-WIFI	1	1	ESP8266	0	0	0	0	0
P-WIFI	1	1	ESP8266	1	0	1	0	0
R-WIFI	1	1	ESP8266	2	0	0	0	0
Q-WIFI	1	1	ESP8266	4	0	1	0	0
N-LORA	1	2	SIM800*	0	0	0	0	0
P-LORA	1	2	SIM800*	1	1	0	0	0
R-LORA	1	2	SIM800*	2	0	0	0	0
Q-LORA	1	2	SIM800*	4	1	0	0	0
W-LORA	1	2	SIM800*	3	2	0	0	0
G-SIM	2	1	SIM800	0	0	0	0	0
G-SIM-WOR	2	1	SIM800	0	0	0	0	1
G-WIFI	2	1	ESP8266	0	0	0	0	0
G-WIFI-WOR	2	1	ESP8266	0	0	0	0	1

*On a LoRa sensor node, the IP selection satisfies the header's modem selection rule but the IP transport is inactive. It does not enable simultaneous SIM/WiFi and LoRa on the shared node UART. Either IP selection can satisfy this rule without creating a different active-radio profile.

Example: controller Sleep, SIM stays normal

```
#define USE_SIM800
#define APP_ROLE APP_ROLE_SENSOR_NODE
#define APP_UPLINK APP_UPLINK_MQTT_DIRECT
#define APP_CONTROLLER_POWER_MODE APP_COMPONENT_MODE_REMOTE_WAKE
#define APP_SIM_POWER_MODE APP_COMPONENT_MODE_NORMAL
#define APP_WIFI_POWER_MODE APP_COMPONENT_MODE_NORMAL
#define APP_LORA_POWER_MODE APP_COMPONENT_MODE_NORMAL
```

For WiFi, select only `USE_ESP8266`. For LoRa, select `APP_UPLINK_LORA_TO_GATEWAY`. The recipe table supplies all other policies. Changing the mode requires a new firmware build; WAKE is an active-time command, not a permanent mode-selection command.

04 | NORMAL and REMOTE_WAKE in detail

NORMAL

The controller follows the configured sampling and reporting rates. The selected radio remains in its normal operating mode. Commands are serviced through the communication-first application path when the link is available. There is no scheduled power-manager Sleep, Stop or Standby transition. The WAKE lease command is unsupported in this profile; normal commands remain available.

Gateway NORMAL runs MQTT and LoRa together and forwards node traffic and commands. Gateway firmware disables local RS485 sensor acquisition. A gateway is not allowed to use any low-power controller profile.

REMOTE_WAKE: light controller Sleep, normal modem

1. Boot, initialize the selected transport and start a fresh sensor/report cycle.
2. After a successful local scheduled send, open the normal command window (default 5 seconds). Repeated sends do not continually extend that window.
3. Once the window ends and the transport is idle, enter REACHABLE_IDLE. Scheduled sensor reads and Status sends pause. The controller uses WFI between service passes.
4. Continue UART processing, MQTT keepalive/reconnect or LoRa polling, and foreground watchdog checkpoints. Service is allowed immediately when UART bytes are present, otherwise at the configured poll interval (default 100 ms).
5. Start an automatic fresh cycle after the reachable-idle interval (default 300 seconds), or accept a server WAKE lease before then.

The 100-ms poll interval is not a 100-ms uninterrupted CPU sleep. SysTick remains enabled and may wake the CPU every millisecond. SIM800 and ESP8266 are not sent modem-sleep commands in this profile; LoRa remains normal receive/transmit Mode 0.

Server WAKE behavior

Send WAKE with `cmd_data=60` to keep normal application work active for 60 seconds; 0 chooses the default lease. Accepted duration is 1..86400 seconds. A newly accepted lease requests a fresh acquisition/report cycle. The immediate command ACK can contain the previous sensor snapshot; the new measurement follows through the scheduler.

An identical retry with the same active request ID is re-ACKed without extending the original lease or requesting another sample. Changed or expired repeats are rejected. Deduplication state is in RAM, not durable replay protection across reset. A new request ID starts a new lease.

Lease expiry does not cancel an in-flight radio operation. A regular periodic cycle stays active if its report fails; a WAKE lease has its own expiry. The automatic idle timer uses `millis()`, not an RTC alarm. Only WAKE starts the explicit active lease; other commands can still be handled while reachable idle.

Trade-off: communication remains available, but the modem, UART and clocks still consume power. Remote availability also depends on network, broker and gateway health; no fixed command-arrival time is guaranteed.

05 | RTC_PERIODIC and RTC_QUEUED in detail

Shared deep-sleep sequence

Both profiles support ESP8266 or LoRa sensor nodes only. They start a fresh acquisition/report cycle after boot and every scheduled wake. After a successful local report, they open a command window, wait for communication activity to finish, prepare the selected radio for deep sleep, and commit a checked sleep record in RTC backup registers 15..19.

The controller then performs a software reset. Very early in setup, before the software-started IWDG is armed, it validates the record, sets the RTC alarm and enters Standby. The alarm wake resets the STM32 again; setup reconnects/reinitializes the radio and starts the next cycle. RAM and temporary runtime settings are not retained by Standby.

- **ESP8266:** close MQTT/TCP, request indefinite `AT+GSLP=0` deep sleep through TinyGSM, end the UART. Wake/restart uses PB8 / ESP_RST and PB7 / ESP_EN during normal initialization, then network and MQTT reconnection.
- **LoRa:** wait for idle/AUX, set M0=1 and M1=1 (Mode 3), end the UART. No RF reception is possible in this mode. Wake initialization restores normal Mode 0.
- **RS485:** flush/end Serial1 and hold PA8 / DE low during handoff. This does not switch off the transceiver or the external sensors' supplies.

RTC_PERIODIC

Default awake command window: 5 seconds. No command can interrupt actual Standby. Ordinary commands must arrive while the node is awake. This profile rejects WAKE leases. The sleep request requires a sane RTC epoch (at least 01 January 2024); unlike RTC_QUEUED, this path does not additionally test the RTC trusted flag. Commission both with STIME anyway.

RTC_QUEUED

Default awake command window: 20 seconds. A trusted, sane RTC is required; send STIME before commissioning sleep. On the server choose **Queue until next Status** for the individual node. Commands are stored durably in `pending_commands.db`. A validated matching Status triggers dispatch of one due command, while the node's receive window is open.

WAKE can be accepted during that awake window and postpones Standby for its lease duration. **It cannot wake a node already in Standby.** A queued WAKE waits for the RTC wake and subsequent Status. The queue is not an MQTT retained-message mechanism.

Queued command expiry defaults to 86400 seconds; supported TTL is 60..604800 seconds. Publication is not execution proof: a matching application ACK confirms the command. Value-setting commands can retry on a later Status after a lost ACK. WAKE is dispatched at most once to avoid starting a fresh lease after a reset; a lost WAKE ACK remains unconfirmed until expiry. More queue entries need later Status opportunities.

Timing and recovery

The Standby alarm deadline is computed from the time the sleep request begins, before radio preparation. Thus the configured interval is not necessarily the exact duration in physical Standby, and report-to-report time also includes command-window, boot, acquisition and reconnection time. Failed reports keep the node awake. Invalid RTC/backup state or failed preparation cancels or delays sleep and resumes/retries safely. These are planned software/Standby resets, not WDT-hang resets.

06 | LORA_WOR_STOP in detail

This profile combines STM32 Stop with a LoRa radio that periodically listens for wake-preamble traffic. It is not LoRa Mode 3 deep sleep and does not support direct SIM800 or ESP8266 wake.

Node sequence

1. Acquire fresh readings and send the scheduled report through the gateway.
2. On successful delivery, open the LoRa command window (default 5 seconds). Set the next report RTC deadline to the delivery-time epoch plus `APP_POWER_SLEEP_SECONDS`.
3. With a trusted RTC, no active WAKE lease and an idle transport, switch LoRa to Mode 2: M0=0, M1=1. End the UART and attach the AUX falling-edge interrupt on PC13.
4. Suspend SysTick and enter STM32 Stop with the low-power regulator. RAM is retained. RTC heartbeat alarms occur no later than every 8 seconds by default, or sooner at the report deadline.
5. After each wake, restore board clocks and the tick before communication. A healthy RTC heartbeat services the watchdog in foreground and returns to Stop; it does not automatically acquire all sensors or upload another Status.
6. LoRa AUX activity restores Mode 0 and opens a receive window for the following command. The scheduled report deadline instead starts a fresh sensor/report cycle. A received WAKE lease postpones further Stop entry.

Required gateway behavior

Set `APP_LORA_GATEWAY_WOR_TX=1` on a separately built gateway. When forwarding a unicast or broadcast command it temporarily selects LoRa Mode 1, sends a wake probe, waits 250 ms, sends the actual command with the wake preamble, then restores Mode 0. The first probe can be deliberately lost while the sleeping node restores its clocks and UART; it is not the application command.

The gateway remains awake with its IP modem normal. Use matching LoRa channel, address/network, UART and RF settings. Check that the actual E32/AS32 variant implements the expected Mode 1/2 wake-preamble behavior and that gateway preamble settings cover the node's RF listening interval. Compiler acceptance alone does not verify a real RF wake.

Hardware and watchdog requirements

- M0 = PB2, M1 = PB1, AUX = PC13. AUX must really reach the MCU and provide the expected interrupt edge.
- The default heartbeat is 8 seconds. The actual guard requires the WDT timeout to be greater than heartbeat + 2 seconds (an exact 2-second margin is rejected). The default WDT is now 30 seconds.
- AUX/RTC interrupt handlers set flags; they do not feed the watchdog. Foreground progress after a normal heartbeat does.
- Failed Mode 2 preparation or unexpected wake/Mode 0 restoration failure aborts Stop and recovers in active mode instead of claiming that a sleeping node is reachable.

Result: this is the implemented profile offering controller Stop plus radio-assisted early command wake. It avoids full reboot on a normal wake, but still has RF duty-cycle, wake-probe and command-delivery latency. Every deployment needs a real-radio bench test.

07 | Configuration timings and actual pins

These are current source defaults, not forced changes to the user's configuration. Compile-time overrides may change them.

Macro	Default	Valid range	Purpose
APP_POWER_SLEEP_SECONDS	300 s	5..604800 s	Standby request interval; REMOTE_WAKE idle interval; WOR report deadline interval
APP_POWER_COMMAND_WINDOW_MS	5000 ms	0..60000 ms	RTC_PERIODIC and REMOTE_WAKE post-send window
APP_QUEUED_COMMAND_WINDOW_MS	20000 ms	5000..60000 ms	RTC_QUEUED post-send window
APP_LORA_WOR_COMMAND_WINDOW_MS	5000 ms	1000..60000 ms	WOR post-send / radio-wake receive window
APP_LORA_WOR_ WATCHDOG_WAKE_SECONDS	8 s	2..15 s	Stop-mode RTC watchdog heartbeat; also needs WDT margin
APP_POWER_SLEEP_ENTRY_TIMEOUT_MS	1000 ms	100..5000 ms	Requested radio-idle / preparation bound; not a cap on every internal AT call
APP_REMOTE_WAKE_DEFAULT_SECONDS	60 s	1..86400 s	Lease when WAKE data is 0
APP_REMOTE_WAKE_POLL_INTERVAL_MS	100 ms	10..1000 ms	Reachable-idle scheduled communication service
APP_LORA_GATEWAY_WOR_TX	0	0 or 1	Gateway wake-preamble / probe option
APP_ENABLE_HARDWARE_WATCHDOG	1	0 or 1	Keep 1 for hang recovery; 0 disables software-started WDT coverage
APP_WATCHDOG_TIMEOUT_US	30000000 us	1..32.76 s	Nominal 30-second timeout without a healthy watchdog checkpoint; macro is in microseconds; WOR needs additional margin
APP_NETWORK_RETRIES_PER_STAGE	10	1..100	SIM/WiFi failure budget per recovery stage

Pins used by the implemented modes

Function	STM32 pin	Meaning
ESP enable / reset	PB7 / PB8	EN active high; reset active low; reset pulse 100 ms, boot wait 3 s
SIM emergency reset	PB0	Open-drain low pulse 250 ms, then release; boot wait 3 s. Not a power switch
LoRa mode / AUX	PB2 / PB1 / PC13	M0 / M1 / AUX; AUX wakes Stop profile
Node selected-radio UART	PA11 TX / PA12 RX	Shared choice: direct IP modem OR LoRa
Gateway LoRa UART	PA9 TX / PA10 RX	Separate from gateway IP modem UART
RS485 transmit enable	PA8	DE low during Standby preparation
Battery measurement	PA0	Not used as Standby wake input; wake pin 1 is disabled on that path

SIM DTR, RI and PWRKEY are not implemented as MCU-controlled sleep/wake signals here. Having DTR/RI on the SIM module does not mean they are routed and supported by this firmware.

08 | Server operation and reported mode fields

Immediate versus queued command

For REMOTE_WAKE, send WAKE using immediate delivery while the node is reachable. For LORA_WOR_STOP, use immediate delivery with an awake WOR-enabled gateway. For RTC_QUEUED, choose queued delivery when the node is in Standby; the next Status allows dispatch. RTC_PERIODIC cannot accept a WAKE lease, though ordinary commands can be handled in its awake window.

Authenticated server API examples (replace the company and node ID with deployed values):

```
{"company":"Company_2","node_id":2,"cmd_type":"WAKE",
 "cmd_data":60,"delivery":"now","broadcast":false}

{"company":"Company_2","node_id":2,"cmd_type":"WAKE",
 "cmd_data":60,"delivery":"next_wake","ttl_seconds":86400,
 "broadcast":false}
```

The command endpoint is `POST /api/command`; queued status is available through `GET /api/queued_command/<request_id>?company=<company>`. Queued delivery targets one node, not a broadcast. Keep the server and broker connected before the next Status; the node's window starts at local transport delivery, not at the moment the browser displays it.

Config numbers are not protobuf mode numbers

Mode name	Controller config value	Reported wire mode
NORMAL	0	1
RTC_PERIODIC	1	2
REMOTE_WAKE	2	3
RTC_QUEUED	4	4
LORA_WOR_STOP	3	5

`power_state=1` means ACTIVE. `power_state=2` means REACHABLE_IDLE and is explicitly reported by REMOTE_WAKE. Other profiles currently return state 1 while reporting; they do not send a live "asleep" state from Standby or Stop. A last displayed ACTIVE packet is a snapshot, not proof that the node is still awake now.

`wake_remaining_seconds` is rounded up at the time the packet is built. `wake_request_id` correlates the lease/ACK. Neither is a guarantee of remaining time when the browser receives the packet. Publication to MQTT or acceptance by the LoRa link is not proof that the web server received a Status; an application ACK is needed to confirm command execution.

Command handling has priority in the application's service path, but an opaque blocking AT or sensor call cannot be preempted by that policy. Link outage or a WDT reset can delay or prevent delivery.

09 | Safety, reset behavior and real power saving

Watchdog and network recovery in every applicable profile

The default independent watchdog is nominally 30 seconds. It monitors foreground software progress, not just network connection. A stalled function that does not reach a valid checkpoint can reset the STM32. It is not fed blindly from UART/RTC interrupt handlers. Actual IWDG timing depends on the device's low-speed clock tolerance.

For the selected active SIM or WiFi transport, 10 returned network failures trigger the selected modem's RST pin. Another 10 failures after that trigger the modem RST plus a fast watchdog controller reset. A successful network connection clears the recovery budget. Broker-only MQTT failures do not count as network failures. A single driver call that exceeds the WDT limit can cause a reset before all 10 retries; the retry counter does not override the watchdog. These are failure recovery, not scheduled sleep modes.

In REMOTE_WAKE, the watchdog remains armed during reachable idle. In LORA_WOR_STOP it remains active and is serviced after healthy RTC heartbeat wakes. Standby uses the early-boot, pre-IWDG handoff. Check the MCU's watchdog option bytes: an always-on hardware watchdog would invalidate the assumption that software can defer starting it until after Standby.

What is NOT automatically powered down

- External TUF/Supmea/HNDA/Sondas sensor power supplies.
- The RS485 transceiver supply, even if the UART is stopped and DE is low.
- Board voltage regulators, power LEDs, dividers and other continuously powered circuitry.
- An unselected modem that is still physically installed and powered.
- External SPI flash through a dedicated deep-power-down command; this power manager does not issue one.

Standby resets RAM-based command leases and temporary sampling/transmission/valve runtime state. Do not assume every server setting persists across it. RTC backup data requires its backup supply/domain to survive. Flash-backed records survive only according to their own commit logic. Standby GPIO state is not an active output guarantee: validate external pull resistors, actuator safety and LoRa M0/M1 levels during reset and Standby.

Current and battery-life claims

No whole-board sleep current has been measured by this software review. There is therefore no validated mA saving or battery-life number in this reference. REMOTE_WAKE saves scheduled work but keeps the modem and clocks alive; Stop plus WOR reduces controller activity while the radio still listens periodically; Standby plus radio deep sleep removes more active functions but needs reboot/reconnection and loses early remote reachability.

Measure the complete node supply, not just the MCU rail, over full active and idle cycles. Average current is `(I_active * T_active + I_idle * T_idle) / (T_active + T_idle)`, adding other wake phases if present. Include network reconnection, RF retries, sensor supplies and regulators. A simulator proves protocol behavior, not radio wake, electrical current or RTC retention.

10 | Step-by-step acceptance tests and evidence

Test one profile at a time. Save the deployed configuration first. Use a 30-second interval for initial commissioning, normal sensor simulator addresses, a connected server and a known-good transport. Send STIME before testing RTC sleep profiles. Restore the intended production interval after testing.

Test	Action	Required observation
Baseline	Run N-SIM, N-WIFI or N-LORA	Fresh Status reaches server; normal commands receive matching replies; no scheduled sleep
CPU Sleep + normal modem	Run R-SIM, R-WIFI or R-LORA; wait for reachable idle	REACHABLE_IDLE log; scheduled sensor polls pause; transport still processes commands
Remote lease	During idle send WAKE 60; retry same request ID	Matching ACK; fresh reading follows; retry does not extend lease; returns idle after expiry and idle transport
Remote periodic cycle	Wait the configured idle interval without WAKE	A fresh acquisition/report cycle starts without full reboot
Periodic Standby	Run P-WIFI or P-LORA after STIME	Command window, radio preparation, software-reset handoff, Standby, RTC reset-wake, fresh report
No premature Standby wake	Send a command during actual P/Q Standby	No immediate reception; P rejects WAKE lease; Q processes queued WAKE only after scheduled wake/Status
Queued lease	Run Q-WIFI or Q-LORA; queue WAKE 60 while asleep	Durable pending item; dispatch after Status; correlated ACK; sleep postponed for lease
Queue expiry / ACK loss	Expire an item; separately suppress one ACK	Expired item is not sent; unconfirmed WAKE is not automatically dispatched again
WOR RTC path	Run W-LORA with trusted RTC	Stop heartbeat recovery; no full setup/reset on normal wake; fresh report at deadline
WOR early command	Use G-SIM-WOR or G-WIFI-WOR; send WAKE before deadline	Mode 1 probe, AUX wake, clock/UART restoration, actual command ACK, active lease
Invalid RTC / busy radio	Exercise untrusted RTC in Q/W, then busy transport	No unsafe sleep; delay/abort log and active recovery; pending command is not hidden
Network recovery	Disconnect SIM/WiFi network during an awake phase	Modem RST after first 10 returned failures; modem + MCU reset after next 10, unless WDT fires earlier
General hang	Use a controlled bench-only stall without checkpoints	Approximately 30-second WDT reset and expected reset-cause/counter report
Power measurement	Record full-cycle supply current for each used profile	Measured active/idle durations and average; external sensors/unused modules included

Source basis and verification boundary

The configuration matrices come from the real `src/config/AppConfig.h` guards. Runtime behavior was checked in `src/power/PowerManager.cpp`, `RemoteWakeCycle.h`, `src/comm/MQTT.cpp` and `src/radio/LoraLink.cpp`. Server queue behavior comes from `server_web.py` and `command_queue.py` in the current web-server folder. All node paths above are relative to `Node/IoT_Node_Encrypted`.

The accompanying `verify_power_combinations.py` preprocesses the actual header against 155 active-controller/radio combinations: 35 node combinations and 120 gateway combinations (including WOR off/on). It expects 15 accepted recipes and 140 rejected combinations. This checks configuration guards, not ARM linking or electrical behavior. The previous release already recorded its 11-profile ARM build matrix under `Documentation/ReleaseVerification_20261004`; that matrix is not a claim that all 15 recipes were built or tested on hardware.